МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ (НИ ТГУ)
Научно-образовательный центр «Высшая ИТ школа»

КУРСОВАЯ РАБОТА

Разработка iOS–приложения для адаптивного планирования и повышения продуктивности

Коротков Глеб Дмитриевич

По направлению подготовки 09.03.04 Программная инженерия

«Программная инженерия»

Руководитель:
должность, учёная степень, учёное звание
_____ / Иванова Л. С.
«____» _____ 2026 г.
Автор работы:
студент группы _____
_____ / Коротков Г. Д.
«____» _____ 2026 г.

Томск – 2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ..... 3
1 Анализ и формализация требований заказчика..... 5
1.1 Постановка задачи заказной разработки..... 5
1.2 Анализ приложений-аналогов..... 5
1.3 Функциональные требования..... 6
1.4 Нефункциональные требования..... 9
1.5 Модель вариантов использования..... 11
1.6 Модель предметной области..... 12
2 Проектирование приложения..... 14
2.1 Выбор платформы и инструментов..... 14
2.2 Выбор UI–фреймворка..... 14
2.3 Архитектура MVVM–C..... 15
2.4 Хранение данных и системные интеграции..... 17
3 Реализация и проверка приложения..... 19
3.1 Проектирование пользовательского интерфейса..... 19
3.2 Реализация приложения..... 20
3.2.1 Экраны входа и регистрации пользователя..... 20
3.2.2 Реализация календаря задач..... 22
3.2.3 Реализация создания задач, событий и подзадач..... 25
3.2.4 Реализация распределения задач..... 27
3.2.5 Обработка переноса задачи после 23:00..... 29
3.2.6 Реализация уведомлений..... 31
3.2.7 Реализация дневника сна и эмоций..... 34
3.2.8 Реализация фокус-сессии и Live Activity..... 35
3.2.9 Реализация подсказки дня и алгоритма анализа..... 35
3.2.10 Реализация статистики, достижений и профиля..... 38
3.3 Проверка соответствия требованиям заказчика..... 38
3.4 Исследование результатов пользовательского тестирования..... 41
ЗАКЛЮЧЕНИЕ..... 47
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ И ЛИТЕРАТУРЫ..... 48
ПРИЛОЖЕНИЕ А..... 51

ВВЕДЕНИЕ

Курсовая работа посвящена заказной разработке мобильного приложения Luma. Заказчиком выступило физическое лицо, с которым автор связался через фриланс-платформу. Персональные данные заказчика и название площадки в работе не раскрываются. Техническим заданием служит совокупность переданных заказчиком функциональных и нефункциональных требований. Они определяют назначение приложения, состав функций и ограничения, которые нужно учесть при проектировании и проверке результата.

Luma предназначено для персонального планирования и объединяет работу с задачами, сведениями о сне и эмоциональном состоянии пользователя. При составлении плана важен не только перечень дел, но и доступное время, текущая нагрузка и состояние человека. Исследования тайм-менеджмента связывают управление временем с результативностью и благополучием [9]. Сон и эмоциональное состояние, в свою очередь, связаны с вниманием, работоспособностью и субъективной оценкой ресурса [6; 7; 10; 11]. Поэтому в требования к приложению вошли автоматическое планирование, дневник сна и эмоций, анализ дня и рекомендации по распределению нагрузки.

Выбор iOS обусловлен содержанием технического задания. Приложение должно взаимодействовать с системным календарём, локальными уведомлениями, Face ID и Live Activity. Нативные средства платформы позволяют обращаться к этим службам в рамках единого приложения и учитывать ограничения конкретного устройства, в том числе отсутствие поддержки отдельной системной возможности или отказ пользователя в разрешении.

В заказной постановке различаются роли заказчика, разработчика и пользователя. Заказчик задаёт ожидаемый результат и границы проекта. Разработчик анализирует и формализует требования, принимает проектные решения, реализует функции и организует их проверку. Пользователь непосредственно работает с приложением: создаёт и планирует задачи, ведёт дневник, запускает фокус-сессии, просматривает анализ, статистику и достижения. Документированная приёмка результата заказчиком в данной работе не заявляется.

Объектом исследования является процесс персонального планирования повседневной деятельности пользователя с учётом его текущей нагрузки, сна и эмоционального состояния.

Предмет исследования — методы и программные средства проектирования и реализации заказного iOS-приложения для адаптивного планирования, контроля выполнения задач и анализа состояния пользователя.

Цель работы состоит в том, чтобы спроектировать и реализовать в соответствии с требованиями заказчика мобильное приложение Luma для iOS, обеспечивающее адаптивное планирование задач, ведение дневника сна и эмоций, проведение фокус-сессий, анализ состояния и отображение статистики пользователя.

Достижение поставленной цели требует решения следующих задач:

1. проанализировать предметную область, переданные заказчиком требования и существующие приложения-аналоги;

2. формализовать функциональные и нефункциональные требования и определить способы их проверки;

3. разработать модели вариантов использования и предметной области;

4. обосновать выбор платформы, инструментов и архитектуры приложения;

5. спроектировать пользовательский интерфейс и структуру хранения данных;

6. реализовать предусмотренные требованиями функциональные модули и интеграции со службами iOS;

7. проверить соответствие реализации требованиям и оценить удобство интерфейса по результатам пользовательского тестирования.

Для решения этих задач применяются анализ требований и приложений-аналогов, объектно-ориентированное моделирование, архитектурное проектирование, программная реализация, инспекция кода, функциональная проверка и пользовательское тестирование интерфейса. Результатом работы являются приложение Luma и комплект проектных материалов, связывающих требования с пользовательскими сценариями, архитектурными решениями, программными модулями и способами проверки. Такая формулировка результата не означает подтверждённой приёмки заказчиком или полного выполнения каждого требования: соответствующие выводы могут делаться только на основании отдельных свидетельств проверки.

1 Анализ и формализация требований заказчика

Исходной точкой проектирования Luma служит техническое задание, представленное переданными заказчиком функциональными и нефункциональными требованиями. На этапе анализа требования были сгруппированы и получили устойчивые идентификаторы, чтобы далее сопоставлять их с вариантами использования, проектными решениями, реализацией и проверкой. Анализ приложений-аналогов дополняет эту работу, но не изменяет границы заказа и не подменяет требования заказчика.

1.1 Постановка задачи заказной разработки

Заказчик ожидает мобильное приложение, которое помогает составлять план дня с учётом уже занятых интервалов, важности, сложности и продолжительности задач, а также сведений о сне и эмоциональном состоянии. В функциональный объём входят учётная запись и профиль, календарное планирование, уведомления, дневник, фокус-сессии, анализ дня, статистика и достижения. Нефункциональные требования задают ограничения по совместимости, доступности интерфейса, производительности, надёжности, защите данных, автономной работе, сопровождаемости и тестируемости.

Основным актором системы является пользователь. Заказчик рассматривается как заинтересованная сторона и источник требований, поскольку его взаимодействие с приложением при эксплуатации в техническом задании не определено. Системные службы iOS выступают поддерживающими акторами только в тех сценариях, где Luma запрашивает разрешение, получает внешние данные или передаёт состояние календарю, службе уведомлений, Face ID либо ActivityKit.

При расхождении текстового описания, UML-модели и реализации приоритет имеет техническое задание. Диаграмма не вводит новую обязательную функцию, а наличие функции в коде само по себе не подтверждает выполнение требования. Обнаруженное расхождение должно быть зафиксировано как ограничение реализации либо как изменение, требующее отдельного решения заказчика.

1.2 Анализ приложений-аналогов

Для сравнения выбраны TickTick, Structured, Todoist и Fabulous. Первые три продукта относятся к планировщикам задач, а Fabulous ориентирован на формирование привычек и повседневных ритуалов. Сопоставление выполнено по функциям, которые связаны с техническим заданием Luma: управление задачами, календарное представление, планирование времени, фокус-сессии, дневник сна и эмоций. Сведения о возможностях приложений проверены по официальным страницам продуктов и их справочным центрам [25–30].

TickTick поддерживает списки задач, календарные представления, напоминания, фильтры, статистику и встроенный таймер Pomodoro. Structured объединяет задачи в визуальной временной шкале, позволяет импортировать календарь, работать с подзадачами и использовать фокус-таймер. Todoist предоставляет задачи, приоритеты, подзадачи, напоминания и интеграцию с календарём. Fabulous строится вокруг привычек, рутин и фокусирующих практик, поэтому не является полноценным диспетчером задач в смысле технического задания Luma.

Официальные описания не подтверждают, что выбранные продукты одновременно предоставляют адаптивное распределение задач с учётом сложности, важности, сна и эмоционального состояния, а также дневники сна и эмоций, анализ дня и систему достижений в одной модели данных. Поэтому обоснование разработки Luma следует строить на сочетании функций и правилах их совместной работы, а не на утверждении, что у аналогов отсутствуют отдельные базовые возможности.

Таблица 1.1 - Сравнение функциональности приложений-аналогов

Критерий	**TickTick**	**Structured**	**Todoist**	**Fabulous**
Управление задачами	Да: задачи, приоритеты, сроки, напоминания	Да: задачи на временной шкале, длительность и сроки	Да: задачи, приоритеты, сроки и напоминания	Частично: привычки и ежедневные ритуалы
Подзадачи	Да	Да	Да	В официальном описании не заявлены
Календарное представление	Да: несколько календарных режимов	Да: временная шкала, недельный и месячный режимы, импорт календаря	Да: календарная интеграция	В официальном описании не заявлено как диспетчер задач
Автоматическое планирование по совокупности важности, сложности и свободного времени	Не подтверждено	Есть средства планирования, но требуемый алгоритм не подтверждён	Не подтверждено	Не подтверждено
Встроенная фокус-сессия с таймером	Да: Pomodoro	Да: фокус-таймер	Не подтверждено как встроенная функция	Да: фокусирующие практики; эквивалентность таймеру Luma не подтверждена
Дневник и анализ сна	Не подтверждено	Не подтверждено	Не подтверждено	Есть вечерние и утренние рутины, но требуемый дневник и анализ сна не подтверждены
Дневник эмоций с интенсивностью и причиной	Не подтверждено	Не подтверждено	Не подтверждено	Не подтверждено
Совместный анализ задач, сна и эмоций	Не подтверждено	Не подтверждено	Не подтверждено	Не подтверждено

Примечание: «не подтверждено» означает, что функция в требуемом виде не найдена в проверенных официальных материалах. Это не равнозначно доказательству полного отсутствия функции во всех версиях продукта.

1.3 Функциональные требования

Функциональные требования описывают действия пользователя и ожидаемые реакции приложения. Для сквозной трассировки детальные положения технического задания объединены в четырнадцать групп FR-01–FR-14. Группировка делает каталог компактным, но не отменяет исходной детализации: обязательные поля, проверки, альтернативные ветви и точные результаты раскрываются в текстовых сценариях.

Таблица 1.2 - Функциональные требования к Luma

ID	Содержание требования	Основной способ проверки
FR-01	Регистрация по имени, дате рождения, электронной почте и паролю; проверка обязательных полей и длины пароля; вход и сохранение сессии.	Позитивные и негативные сценарии регистрации и входа, повторный запуск.
FR-02	Вход через Face ID с резервным входом по паролю; просмотр профиля, выбор аватара, настройка Face ID и уведомлений, выход.	Проверка на устройствах с доступной и недоступной биометрией, отказов и выхода с подтверждением.
FR-03	Навигация по пяти разделам; выбор даты; просмотр прогресса; сортировка и фильтрация задач и подзадач; изменение, завершение и удаление собственных задач.	Проверка переходов, дат, порядка элементов, фильтров и операций с задачами.
FR-04	Совместное отображение собственных задач и доступных только для чтения событий календаря iOS; запрос разрешения и обновление внешних данных.	Проверка предоставления, отказа и отзыва разрешения, изменения внешнего события.
FR-05	Создание события, обычной и сложной задачи; обязательные название, важность и сложность; период или точное время; минимальный блок 15 минут; минимум одна подзадача для сложной задачи.	Валидные и ошибочные данные для каждого типа элемента.
FR-06	Автоматический выбор свободного интервала обычной задачи без пересечений с учётом занятости, важности, сложности и продолжительности.	Наборы занятых интервалов, разные приоритеты и отсутствие свободного окна.
FR-07	Распределение подзадач сложной задачи внутри общего периода без пересечений и формирование итогового плана.	Сценарии с одной и несколькими подзадачами, проверка границ периода и порядка.
FR-08	Планирование преимущественно в пределах 08:00–23:00; предложение разделить обычную задачу при выходе за границу дня и перенос остатка только с согласия пользователя.	Отказ и согласие на разделение, сохранение суммарной длительности частей.

ID	Содержание требования	Основной способ проверки
FR-09	Настройка локальных уведомлений; напоминания о задачах, подзадачах и дневниках; отмена лишних напоминаний; история доставленных уведомлений в обратном хронологическом порядке.	Разрешение, расписание, отмена и сортировка истории.
FR-10	Дневник эмоций с интенсивностью от 1 до 10 и необязательной причиной; ручной ввод времени сна и пробуждения с расчётом длительности; привязка записей к пользователю и дню.	Границы интенсивности, пустая причина, расчёт через полночь и изоляция записей.
FR-11	Фокус-сессия для незавершённой задачи текущего дня: длительность по задаче, музыка либо тишина, таймер, сохранение результата, завершение задачи и условное отображение в Live Activity.	Таймер, аудио, сохранение, завершение и работа на устройстве без поддержки Live Activity.
FR-12	Анализ задач, сна, эмоций, просрочек и нагрузки следующего дня; оценки ресурса, нагрузки и баланса по шкале 0–100; категория, точность и не более четырёх рекомендаций.	Воспроизводимые наборы данных, граничные значения и недостаток данных.
FR-13	Общая статистика за семь дней и подробная статистика эмоций, сна и задач, включая состав задач выбранного дня.	Проверка периодов, агрегатов и перехода к данным выбранного дня.
FR-14	Обновление достижений; состояния «заблокировано», «разблокировано» и «открыто»; 50 единиц опыта за открытие, новый уровень после каждых 250 единиц опыта; уровень и звание.	Переходы состояний, однократное начисление опыта и границы уровня.

Группы FR-01–FR-14 далее раскрываются десятью тематическими UML-изображениями. Они показывают внешних акторов и пользовательские цели, тогда как обязательные данные, проверки, вычисления и альтернативные ветви задаются текстовыми сценариями вариантов использования.

Процесс создания элементов календаря может быть дополнительно раскрыт диаграммой активности: событие сохраняется на выбранное время, обычная задача передаётся алгоритму поиска свободного интервала, а сложная задача сначала декомпозируется минимум на одну подзадачу. Такая диаграмма поясняет FR-05–FR-08, но не заменяет точных формулировок и сценариев проверки.

1.4 Нефункциональные требования

Нефункциональные требования определяют проверяемые свойства приложения и ограничения его реализации. Они не оформляются как варианты использования, поскольку описывают не самостоятельные цели пользователя, а качество работы системы. Для их подтверждения применяются инструментальные замеры, испытания на устройствах,

проверка разрешений и автономного режима, архитектурная инспекция и анализ автоматизированных тестов.

Таблица 1.3 - Нефункциональные требования к Luma

ID	Требование	Способ проверки
NFR-01	Нативное приложение для iOS; корректное отображение на заявленных iPhone и iPad; отсутствие Face ID, Live Activity или разрешения календаря не блокирует остальные функции.	Испытания на заявленных конфигурациях и при недоступности системных возможностей.
NFR-02	Русскоязычный, читаемый и последовательный интерфейс; достаточная контрастность и размеры элементов; безопасные области; состояние передаётся не только цветом.	Инспекция интерфейса, проверка доступности и пользовательское тестирование.
NFR-03	Локальные переходы выполняются не более чем за 1 с, анализ — не более чем за 2 с; длительные операции не блокируют интерфейс; таймер обновляется каждую секунду.	Инструментальные замеры на согласованном наборе данных.
NFR-04	Данные сохраняются между запусками; ошибка операции не повреждает прежние записи; обновление не создаёт дубликаты; миграция не приводит к потере данных.	Тесты сохранения, повторного обновления, отказа и миграции.
NFR-05	Данные изолированы по владельцу; подзадача связана с родительской задачей; состояние согласовано между модулями; дата и время учитывают календарь и часовой пояс.	Проверка связей хранилища, нескольких учётных записей и часовых поясов.
NFR-06	Пароль не хранится открытым текстом; используются криптографическое хеширование с индивидуальной солью и Keychain; Face ID работает системными средствами; после выхода нужна повторная авторизация.	Инспекция хранилища и кода, сценарии входа, биометрии и выхода.
NFR-07	Разрешения запрашиваются только для выбранной функции; задачи, сон, эмоции и анализ хранятся локально и не передаются внешним сервисам без отдельного согласия.	Проверка запросов разрешений, сетевой активности и изоляции данных.
NFR-08	Основные функции и музыка фокус-сессии доступны без Интернета; таймеры, аудио, Live Activity, наблюдатели и фоновые операции корректно завершаются.	Автономный прогон и профилирование жизненного цикла ресурсов.
NFR-09	Интерфейс, бизнес-логика, навигация, хранение и системные службы разделены; зависимости предоставляются через протоколы и внедрение зависимостей.	Архитектурная инспекция модулей и направлений зависимостей.
NFR-10	Алгоритмы планирования, анализа, статистики, уровней и достижений проверяются отдельно от интерфейса; основные сценарии покрываются модульными и интерфейсными тестами.	Анализ тестовой архитектуры, покрытия и результатов полного прогона.

Приведённые способы проверки обозначают необходимые свидетельства и не утверждают, что соответствующие испытания уже выполнены.

1.5 Модель вариантов использования

Модель вариантов использования построена по функциональным требованиям и показывает внешние роли и цели, доступные им в Luma. Пользователь является ведущим актором. Службы календаря, уведомлений, Face ID, медиатеки и ActivityKit показываются как поддерживающие акторы только там, где они фактически участвуют во взаимодействии. Внутренние компоненты приложения — планировщик, хранилище, модели представления и сервисный слой — акторами не являются.

Чтобы сохранить читаемость модели, группы требований раскрываются десятью тематическими UML-изображениями: регистрация и вход; профиль; календарь и управление задачами; создание и планирование элементов; статистика; уведомления; дневник эмоций и сна; фокус-сессия и Live Activity; анализ дня и рекомендации; достижения и уровни. На каждом изображении варианты использования располагаются внутри границы системы Luma, а пользователь и внешние службы iOS — за её пределами.

Диаграммы дают обзор пользовательских целей, но не заменяют точных условий. Детали задаются текстовыми сценариями, в которых фиксируются идентификатор и цель, основной актор, предусловия, событие запуска, основной поток, альтернативы и ошибки, а также успешный результат. В частности, в сценариях должны быть описаны обязательные поля регистрации, резервный вход по паролю, режим чтения событий iOS, правила автоматического планирования и разделения задачи после 23:00, расписание и отмена уведомлений, поля дневника, работа устройств без Live Activity, входные данные анализа и переходы состояний достижений.

Выбор незавершённой задачи и музыкального режима относится к шагам сценария фокус-сессии, а не к независимым целям пользователя. Аналогично проверки полей, сохранение, сортировка, вычисление длительности и обновление прогресса описываются внутри соответствующих сценариев. Блокирование сторонних приложений в переданные требования не входит и поэтому не включается в обязательный функциональный объём.

Автоматическое планирование является общим обязательным поведением при создании обычной и сложной задачи. Предложение разделить обычную задачу при выходе за 23:00 представляет собой условную ветвь: при отказе исходное действие не должно незаметно изменяться, а перенос оставшейся части допускается только после согласия пользователя. Для дневника причина эмоции остаётся необязательной при любом выбранном значении, а сведения о сне вводятся пользователем вручную. Для фокус-сессии

взаимодействие с ActivityKit условно и не должно препятствовать основной функции на неподдерживаемом устройстве.

Десять тематических UML-изображений связывают группы FR-01–FR-14 с внешними ролями, а текстовые сценарии уточняют данные, проверки, вычисления и альтернативные потоки. Модель можно использовать как основу для проектирования и проверки, но она не заменяет текстовую спецификацию.

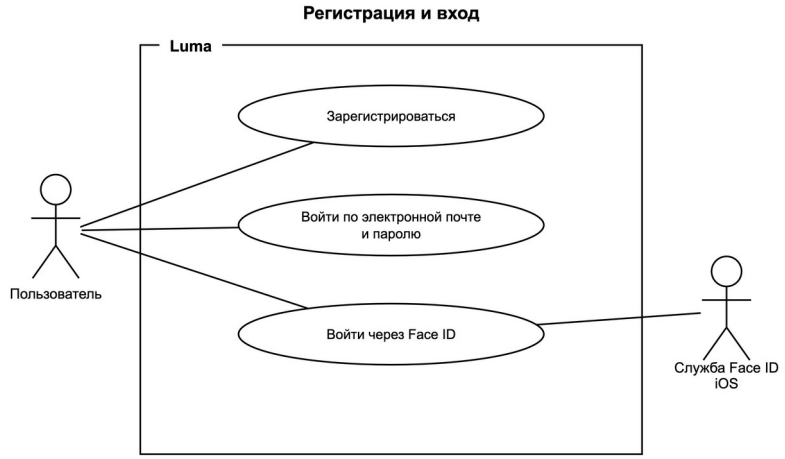


Рисунок 1.1 - Регистрация и авторизация

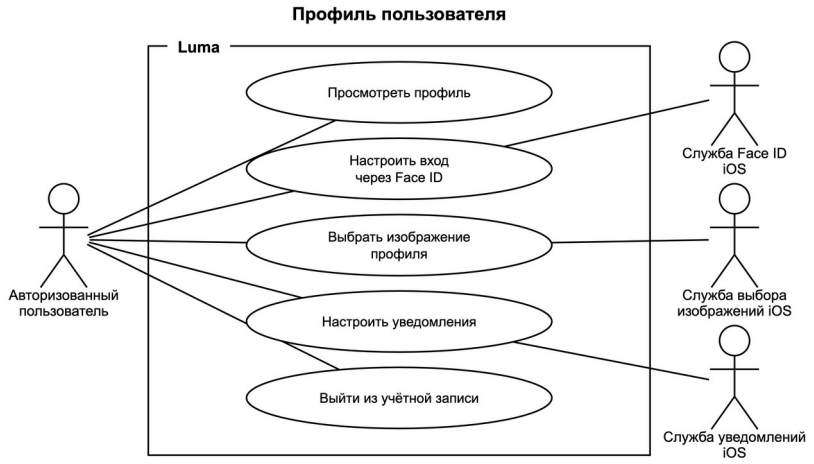


Рисунок 1.2 - Профиль пользователя

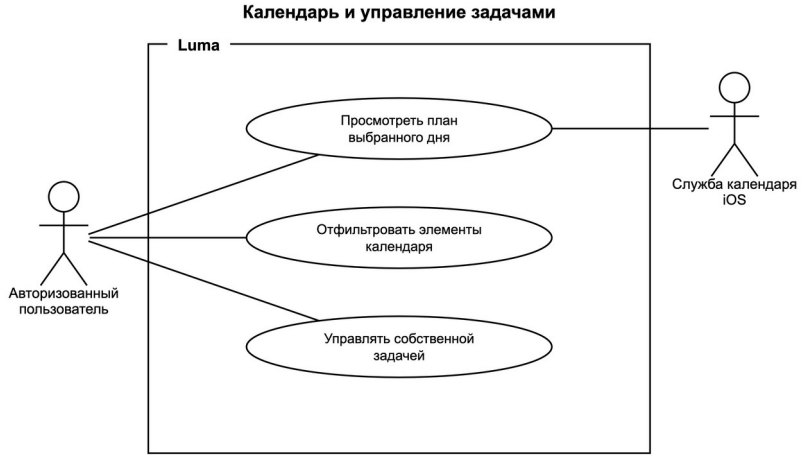


Рисунок 1.3 - Календарь и управление задачами



Рисунок 1.4 - Создание задач и автоматическое планирование

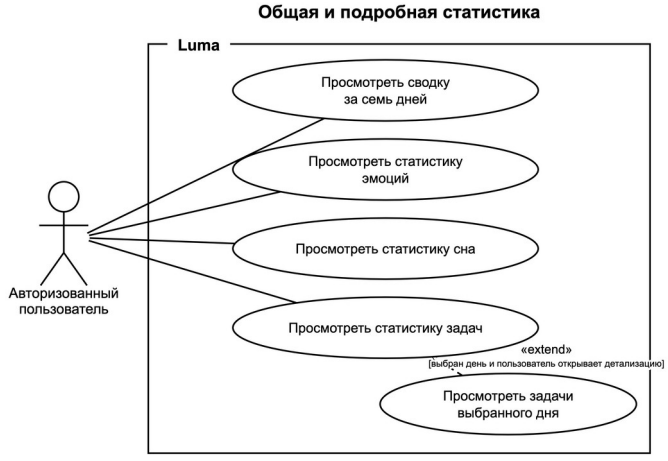


Рисунок 1.5 - Общая и подробная статистика



Рисунок 1.6 - Настройка и получение уведомлений

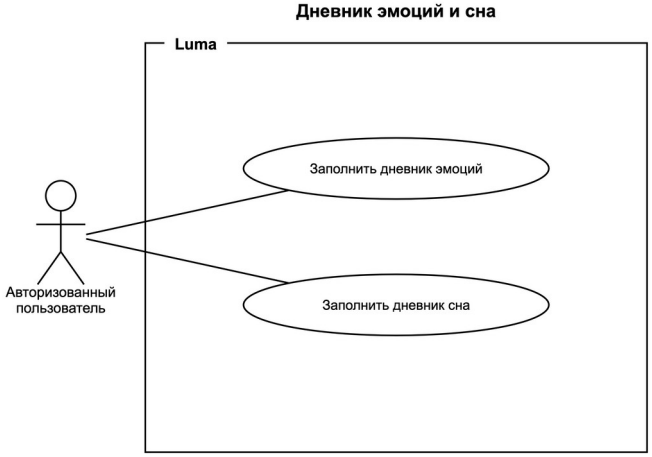


Рисунок 1.7 - Дневник эмоций и сна

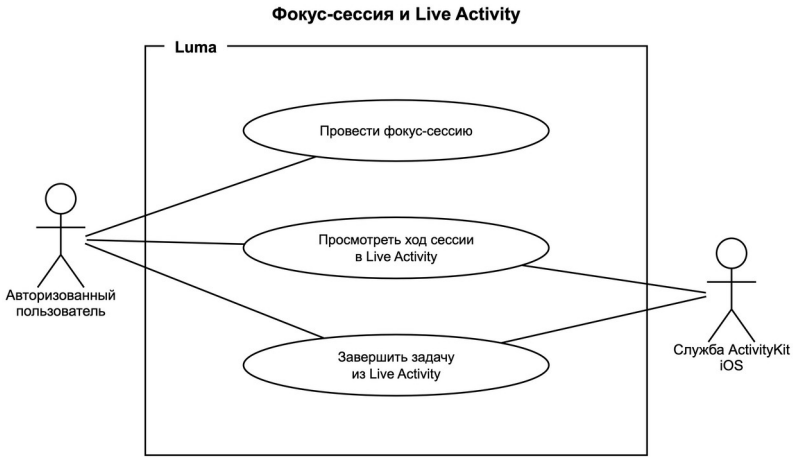


Рисунок 1.8 - Фокус-сессия и Live Activity



Рисунок 1.9 - Анализ состояния пользователя

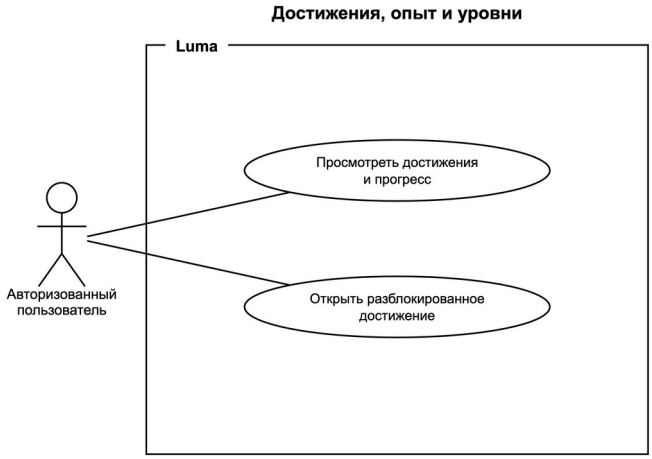


Рисунок 1.10 - Достижения, опыт и уровни

1.6 Модель предметной области

После уточнения функциональных требований и пользовательских сценариев строится модель предметной области Luma. Она фиксирует основные понятия, которыми оперирует приложение, и связи между ними, не подменяя классовую структуру конкретной реализации.

Центральным понятием модели является пользователь. С ним связаны создаваемые задачи, записи сна и эмоционального состояния, фокус-сессии, результаты анализа и достижения. Такая связь выражает принадлежность данных конкретной учётной записи и поддерживает требование об их изоляции; правила удаления и жизненного цикла должны определяться отдельно и не следуют автоматически из самой ассоциации.

Задача представлена в нескольких разновидностях: обычная задача, сложная задача и подзадача. Сложная задача объединяет одну или несколько подзадач, а каждая подзадача относится к своей родительской задаче. Для планирования используются период выполнения, предполагаемая продолжительность, важность и сложность; для события с

фиксированным временем сохраняются начало и окончание. Внешнее событие календаря iOS остаётся доступным только для чтения и учитывается как занятый интервал.

Записи сна и эмоций относятся к пользователю и календарному дню и служат входными данными для статистики и анализа состояния. Результат анализа хранит рассчитанные показатели и сведения, позволяющие определить, сохранилась ли актуальность исходных данных. Повторное использование результата допустимо, пока связанные задачи и дневниковые записи не изменились. Такое представление поддерживает FR-12 и требование NFR-03 к времени выполнения анализа.

Модель предметной области задаёт общий словарь следующей главы: на её основе можно обосновать структуру хранения, границы модулей и взаимодействие компонентов, не смешивая предметные сущности с экранными классами и системными службами iOS.

2 Проектирование Luma

Проектные решения для Luma опираются на требования к платформе, интерфейсу, локальному хранению, системным интеграциям и автономной работе. Отдельно учтено разделение ответственности между компонентами. Глава описывает устройство решения и его связь с требованиями. Совместимость, производительность, надёжность, безопасность и удобство использования оцениваются по результатам сборки, испытаний и измерений.

2.1 Выбор платформы и средств разработки

Техническое задание определяет Luma как нативное приложение для iOS и требует поддержки iPhone и iPad, использования стандартных возможностей системы и сохранения основных функций при недоступности отдельных разрешений. Для реализации выбран Swift. Основная часть пользовательского интерфейса построена на UIKit, экран достижений — на SwiftUI, а отображение фокус-сессии вне приложения вынесено в отдельный target FocusLiveActivityExtension, использующий ActivityKit и SwiftUI. В настройках проекта указаны семейства устройств iPhone и iPad и минимальная версия iOS 26.2.

Выбранные средства соответствуют требуемым интеграциям. EventKit читает события системного календаря, UserNotifications создаёт локальные напоминания, LocalAuthentication выполняет биометрическую проверку, а ActivityKit отвечает за Live Activity. AVFoundation воспроизводит локальную музыку, Core Data хранит данные, а Security и CryptoKit используются для работы с учётными данными. UIKit выбран основным UI-фреймворком, поскольку на нём реализованы экраны, формы и навигационные сценарии. SwiftUI применяется локально: через UIHostingController для AchievementsView и непосредственно в расширении Live Activity. В остальных функциональных разделах два способа построения интерфейса не смешиваются.

Выбор нативных фреймворков создаёт прямой путь к системным API и не вводит обязательной сетевой зависимости. Однако из состава проекта следует только выбранный способ реализации. Корректность интерфейса на разных iPhone и iPad, поведение на устройствах без доступной биометрии или Live Activity, а также устойчивость к отказу и отзыву разрешений требуют проверки на матрице устройств и в отрицательных сценариях. Указанный deployment target также означает, что текущая конфигурация не подтверждает совместимость с версиями iOS ниже 26.2.

2.2 Архитектура и границы модулей

Требования к сопровождаемости предусматривают разделение интерфейса, прикладной логики и хранения данных, вынос навигации из экранов, применение протоколов и передачу зависимостей. Для их реализации выбрана практическая

разновидность MVVM-C с сервисным и репозиторным слоями. Это описание отражает текущий код; отдельного слоя UseCase и классов наподобие CreateTaskUseCase в проекте нет.

Слой представления расположен в каталоге Modules. ViewController и пользовательские View отвечают за отображение и передачу действий, а ViewModel хранит состояние конкретного сценария и вызывает прикладные сервисы. Так, CalendarViewModel загружает, фильтрует и обновляет задачи через TaskService, а FocusViewModel формирует список задач фокусировки и управляет воспроизведением через интерфейс FocusMusicPlaying. Переходы вынесены в координаторы: AppCoordinator выбирает стартовый поток, AuthCoordinator управляет авторизацией, TabBarCoordinator формирует основные разделы, CalendarCoordinator открывает календарные сценарии, а AddTaskCoordinator ведёт пользователя от формы до результата планирования. Такое разделение не даёт смешать построение экрана, переходы и работу с данными в одном контроллере.

Прикладные операции сосредоточены в AuthService, TaskService, DiaryService, AnalysisService и AchievementService. Доменные структуры, валидаторы, расчёт анализа и каталог достижений размещены в Domain. Доступ к Core Data описан протоколами UserRepositoryProtocol, TaskRepositoryProtocol, DiaryRepositoryProtocol и AchievementRepositoryProtocol, которые реализуют одноимённые репозитории без суффикса Protocol. DIContainer создаёт общие экземпляры прикладных сервисов и передаёт их координаторам, а те — моделям представления. Такая схема задаёт явные точки сборки зависимостей и позволяет подменять часть нижележащих компонентов через конструкторы.

Архитектура является слоистой логически, но каталоги App, Modules, Domain и Data не оформлены как отдельные Swift-модули. Кроме того, изоляция системных API выполнена не полностью: например, CalendarEventService и FocusMusicPlayer имеют протоколы, тогда как LocalNotificationService, FocusLiveActivityManager, KeychainManager и AvatarStore используются как конкретные классы или singleton-объекты; некоторые модели представления импортируют EventKit, UserNotifications или LocalAuthentication напрямую. Поэтому корректно утверждать, что выбранная структура уменьшает часть связности и создаёт точки для тестовой подмены. Сопровождаемость, расширяемость и тестируемость всей системы не доказаны самим наличием слоёв: для такого вывода нужны архитектурные проверки и тесты с заменой зависимостей.

2.3 Хранение данных и системные интеграции

Требования к автономной работе, целостности и конфиденциальности предполагают локальное хранение пользовательских данных, связь записей с владельцем и отсутствие обязательной передачи данных внешним сервисам. Для этого выбран NSPersistentContainer и репозитории Core Data. В хранилище представлены UserEntity, TaskEntity, SleepEntryEntity, EmotionEntryEntity, FocusSessionEntity и AchievementEntity. Задачи, записи дневника, фокус-сессии и достижения связываются с UserEntity отношением owner; подзадачи представлены тем же типом TaskEntity и связаны с родительской задачей отношением parentTask. Репозитории фильтруют выборки по идентификатору владельца, а TaskRepository отдельно обрабатывает создание и удаление подзадач.

Выбор Core Data связан с необходимостью хранить взаимосвязанные локальные объекты и получать выборки по пользователю и временному интервалу. PersistenceController включает автоматическую лёгкую миграцию и откатывает контекст при ошибке сохранения. При этом метод save() не возвращает ошибку вызывающему коду. Из-за этого текущая граница репозитория ещё не реализует требование о показе пользователю причины неудачного сохранения. Ошибку необходимо передавать из PersistenceController через репозиторий и прикладной сервис до модели представления.

Для учётных данных выбран отдельный путь хранения. UserRepository сохраняет профиль в Core Data, а KeychainManager вычисляет SHA-256 от пароля и строки идентификатора пользователя и помещает результат в Keychain. LocalAuthentication применяется для проверки Face ID системными средствами. Эти решения соответствуют направлению требований о недопустимости хранения пароля в открытом виде и об использовании системного биометрического механизма. Вместе с тем стойкость выбранной схемы хеширования, защита сессии и отсутствие доступа к данным после выхода должны подтверждаться анализом угроз и отдельными тестами; одно чтение кода не доказывает безопасность.

Системный календарь подключён через CalendarServiceProtocol и CalendarEventService. Сервис запрашивает доступ к EventKit, преобразует полученные события в TaskItem только для чтения и при отсутствии разрешения возвращает пустой список, поэтому внутренние задачи могут обрабатываться без календарных событий. Напоминания создаёт LocalNotificationService; музыка воспроизводится FocusMusicPlayer; Live Activity запускает и завершает FocusLiveActivityManager; изображение профиля сохраняет AvatarStore, предварительно уменьшая его до 400 × 400 точек в фоновой очереди. Эти компоненты выбраны для реализации требований к календарю, уведомлениям, фокус-сессии, локальным ресурсам и обработке изображения.

По исходному коду данные и музыкальные ресурсы обрабатываются локально, а обязательных сетевых клиентов не выявлено. Это создаёт предпосылку для автономной работы, но не заменяет испытание без сети. Аналогично, необходимо отдельно проверить отзыв разрешения календаря, отмену уведомлений после выхода, завершение таймера, музыки и Live Activity, миграцию хранилища, изоляцию двух учётных записей и потребление памяти при работе с изображениями.

2.4 Проектирование пользовательского интерфейса

Требования к интерфейсу задают единый визуальный стиль, постоянный доступ к основным разделам, понятные формы, сообщения об отсутствии данных и ошибках, подтверждение значимых действий и русский язык. Для реализации выбран единый контейнер MainTabBarController с пользовательской панелью CustomTabBar. TabBarCoordinator формирует пять основных разделов: календарь, дневник, общий анализ, подробную статистику и профиль. Переход к ним выполняется выбором соответствующего элемента панели, а вложенные сценарии открываются координаторами функциональных разделов.

Повторяющиеся элементы вынесены в компоненты CardContainerView, CustomButton, CustomTextField, FormFieldCard, DatePickerCard, MetricSliderCard, ValidationHintView и EmptyStateView. Цвета и часть визуальных параметров собраны в расширениях DesignTokens. Формы создания задач используют TaskValidator и SubtaskValidator, а формы авторизации — LoginValidator и RegistrationValidator. Auto Layout и привязка ряда экранов к safeAreaLayoutGuide выбраны для адаптации компоновки и исключения наложения на системные области.

Перечисленные решения учитывают требования на уровне проектирования, но единообразие и доступность всего интерфейса требуют отдельной проверки. В коде присутствуют отдельные accessibility-метки и поддержка Dynamic Type на элементах экрана анализа, однако во многих компонентах заданы фиксированные размеры шрифта и высоты. Поэтому читаемость при увеличенном системном шрифте, достаточная контрастность, размеры зон нажатия, передача состояния не только цветом, работа VoiceOver и компоновка на iPad должны проверяться отдельным аудитом и UI-тестами. До получения таких результатов следует писать «решение выбрано для выполнения требований к доступности», а не «интерфейс обеспечивает доступность».

3 Реализация и проверка приложения Luma

В результате разработки получено готовое мобильное приложение Luma. Глава показывает, как требования технического задания отражены в программных модулях и какие проверки уже выполнены. Для каждой группы требований сначала рассматривается реализация, затем приводится свидетельство и указываются границы вывода. Это позволяет отделить наличие программного механизма от результата его испытания. Успешная компиляция приложения сама по себе не подтверждает полное соответствие техническому заданию.

Фактическая архитектура Luma включает представления UIKit и SwiftUI, модели представления, координаторы навигации, сервисы, репозитории и доменные модели. Отдельного слоя классов UseCase, упоминавшегося в ранней редакции работы, в текущем исходном коде нет. Пользовательское действие принимается контроллером или SwiftUI-представлением, передаётся модели представления, а та обращается к соответствующему сервису либо репозиторию. Зависимости основных модулей создаются контейнером внедрения зависимостей, хотя часть системной инфраструктуры доступна через singleton. Поэтому далее используются только реально существующие имена компонентов.

3.1 Учётная запись, профиль и сессия

Раздел связан прежде всего с FR-01 и FR-02, а также с ограничениями безопасности NFR-06. Пользователь должен зарегистрироваться, указав имя, дату рождения, адрес электронной почты и пароль, пройти проверку обязательных полей и минимальной длины пароля, войти по электронной почте и паролю и восстановить сохранённую сессию при следующем запуске. В профиле должны отображаться сведения текущей учётной записи, аватар, уровень и опыт; там же размещаются настройки Face ID и уведомлений и подтверждаемое действие выхода. Биометрическая возможность должна быть скрыта на неподдерживаемом устройстве, а неудачная проверка Face ID не должна лишать пользователя обычного входа.

Формы регистрации и входа реализованы отдельными модулями RegistrationViewController/RegistrationViewModel и LoginViewController/LoginViewModel. Проверка значений вынесена в RegistrationValidator и LoginValidator: до обращения к хранилищу проверяются обязательность полей, формат введённых данных и длина пароля не менее шести символов. После успешной проверки AuthService вызывает UserRepository. При регистрации репозиторий создаёт локального пользователя, а при входе ищет запись по нормализованному адресу электронной почты и проверяет учётные данные.

Пароль не записывается в модель Core Data. KeychainManager вычисляет SHA-256-хеш из пароля и индивидуальной соли, в качестве которой используется UUID пользователя, и сохраняет полученное значение в Keychain. При входе хеш рассчитывается повторно и сравнивается с сохранённым. Такой механизм соответствует требованию не хранить открытый пароль, но UUID является публичным идентификатором, а не отдельным секретом. Кроме того, результат операции Keychain не поднимается в доменный слой как типизированная ошибка, поэтому статическая инспекция подтверждает используемый способ хранения, но не всю отказоустойчивость регистрации.

Состояние входа хранит SessionManager. При запуске AppCoordinator выбирает один из трёх маршрутов: восстановление основной сессии без повторного ввода, попытка биометрической разблокировки или показ экрана авторизации. Включение Face ID выполняет ProfileViewModel: сначала проверяется наличие именно Face ID через LocalAuthentication, затем запускается системная биометрическая проверка, и только после успеха настройка связывается с текущим идентификатором пользователя. Если технология недоступна или проверка неуспешна, настройка остаётся выключенной, а обычная форма входа остаётся доступной.

Такое разделение важно для границ сессии. Сохранённый идентификатор пользователя сам по себе не заменяет биометрическую проверку, если для учётной записи включён Face ID: координатор сначала открывает отдельный экран разблокировки. При отключённой биометрии сохранённая сессия может быть восстановлена напрямую. После выхода SessionManager удаляет состояние активного пользователя, поэтому повторный доступ к данным проходит через авторизацию. В коде также проверяется, что сохранённый для Face ID идентификатор относится к текущей учётной записи; это предотвращает использование настройки другой локальной записи.

Профиль загружает имя, адрес электронной почты, маску пароля, состояние системных настроек, аватар и рассчитанный прогресс уровня. Изменение аватара поддерживается через системный выбор изображения и AvatarStore; перед сохранением изображение уменьшается. Редактирование имени, адреса электронной почты или пароля в текущей реализации отсутствует, поэтому выражение «изменение профиля» здесь относится только к аватару и настройкам. Выход выполняется после подтверждения, очищает активную сессию, возвращает пользователя на экран авторизации и отменяет ожидающие и уже доставленные локальные уведомления.

Ключевая последовательность авторизации выражена коротким фрагментом сервиса:

swift func login(email: String, password: String) throws -> User { do { return try userRepository.verifyCredentials(email: email, password: password) } catch { throw AuthError.invalidCredentials } }

Листинг 3.1 — Передача проверки учётных данных из AuthService в репозиторий

Листинг показывает фактическую границу ответственности: модель представления обрабатывает действие экрана, сервис задаёт сценарий входа и преобразует ошибку, а репозиторий выполняет поиск пользователя и проверку хеша. Отдельный объект варианта использования в этой цепочке не создаётся.

Регистрация, вход, проверка полей и восстановление сессии прослежены статически, поэтому FR-01 имеет статус «подтверждено кодом». FR-02 подтверждено частично: загрузка профиля, аватар, настройки, условное отображение Face ID и выход реализованы, но реальный Face ID требует проверки на поддерживаемом устройстве, а изменение имени, электронной почты и пароля не предусмотрено. NFR-06 подтверждено частично из-за непроверенных отказов Keychain и отсутствия общей транзакции между созданием записи пользователя и сохранением секрета. Успешная сборка доказывает компилируемость этих модулей, но не прохождение позитивных и негативных сценариев входа.

3.2 Календарь и управление элементами

FR-03 задаёт пять основных разделов Luma, календарный выбор даты, сортировку элементов по времени, фильтры «Все», «Выполняемые» и «Завершённые», вычисление дневного прогресса и операции изменения, завершения и удаления собственных задач. FR-04 требует совместно показывать внутренние задачи и события календаря iOS, запрашивать разрешение непосредственно перед доступом и обновлять список после изменения EventKit либо возврата приложения в активное состояние. Внешние события должны участвовать в расписании, оставаясь доступными только для чтения.

TabBarCoordinator создаёт пять разделов: календарь, дневник, анализ дня, статистику и профиль. Экран календаря передаёт выбранную дату и фильтр в CalendarViewModel. Модель представления получает данные через TaskService, после чего передаёт представлению отфильтрованный список и пару значений для прогресса. При расчёте сложная задача раскрывается до подзадач: завершённые подзадачи участвуют в числителе, все подзадачи — в знаменателе. Этот же принцип применяется при фильтрации, поэтому сложная задача не скрывает состояние отдельных этапов.

Собственные элементы загружаются из TaskRepository, а события iOS — через CalendarEventService, использующий EventKit. CalendarEventTranslator преобразует внешнее событие в общую доменную модель TaskItem и помечает источник как календарный. TaskService объединяет оба массива и сортирует их по фактическому времени

начала. Преобразование не переносит внешнюю запись в Core Data и не предоставляет методов записи в EventKit. В интерфейсе проверяется источник: редактирование системного события не открывается, тогда как для внутренней задачи доступны изменение названия, описания, важности и сложности, переключение выполнения и удаление.

Общая доменная модель упрощает визуальное объединение, но источник сохраняется как обязательное различие поведения. Внутренняя задача имеет владельца, состояние выполнения и операции репозитория. Внешнее событие выступает временным ограничением: его границы используются при отображении и автоматическом планировании, но жизненным циклом управляет системный календарь. Благодаря этому отказ в разрешении EventKit уменьшает только состав объединённого списка и не меняет сохранённые в Core Data задачи пользователя.

Обновление организовано событиями. CalendarViewModel наблюдает собственное уведомление об изменении хранилища задач, EKEventStoreChanged и UIApplication.didBecomeActiveNotification. Серия близких событий сворачивается задержкой 0,2 секунды, после чего список перечитывается. При отказе в доступе к системному календарю сервис возвращает пустой набор внешних событий, поэтому внутренний календарь продолжает работать.

В качестве ключевого свидетельства для раздела следует использовать экран календаря, на котором одновременно видны внутренняя задача и событие iOS, а также дневной прогресс и выбранный фильтр.

FR-03 подтверждено кодом: найдены навигация, выбор даты, фильтры, разворачивание подзадач, расчёт прогресса и операции над собственными задачами. FR-04 подтверждено частично. Режим чтения и обработчики обновления видны в коде, однако системный диалог разрешения, отзыв доступа и фактическая доставка уведомления EventKit требуют runtime-проверки. Создание, изменение и удаление записей системного календаря в приложении не реализуются и не должны заявляться. Неперехваченные ошибки операций toggleTask и deleteTask подавляются конструкцией try?, поэтому информирование пользователя при сбое хранения также требует доработки.

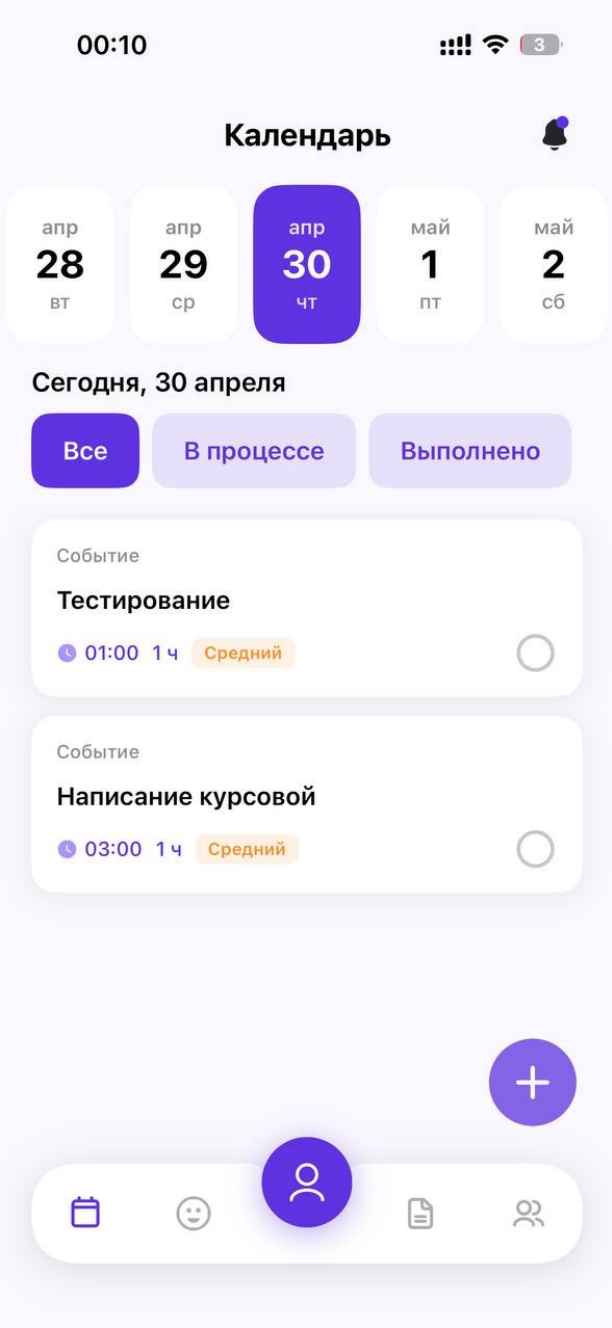


Рисунок 3.1 - Календарь и собственные элементы пользователя

3.3 Автоматическое планирование и граница 23:00

FR-05–FR-07 определяют три создаваемых вида элементов: событие с фиксированным временем, обычную задачу с автоматическим выбором интервала и сложную задачу минимум с одной подзадачей. Для элемента обязательны название, важность и сложность; задача дополнительно получает период и предполагаемую длительность, а подзадача — собственные параметры. Планировщик должен учитывать внутренние задачи и системные события, исключать пересечения и по возможности помещать важные и сложные дела раньше. FR-08 задаёт преимущественный рабочий

диапазон 08:00–23:00 и требует спрашивать согласие пользователя перед переносом остатка задачи на следующий день.

Выбор вида элемента хранит AddTaskFlowViewModel, а формы собраны в модуле AddTask. Для события используются введённые пользователем начало и конец: оно сохраняется без автоматического переноса и затем рассматривается как занятый интервал. Для обычной и сложной задач TaskService сначала получает внутренние элементы за период, затем добавляет считанные из EventKit интервалы и передаёт общий набор алгоритмам из TaskScheduling.swift. Введённая длительность нормализуется минимум до 15 минут при вычислении плана.

TaskAutoScheduler строит рабочие интервалы каждого дня с 08:00 до 23:00, вычитает пересекающиеся занятые отрезки и формирует допустимые кандидаты. Оценка кандидата учитывает день периода, загруженность, важность, сложность и более раннее размещение. Выбирается кандидат с максимальной оценкой. Если внутри рабочих сегментов подходящего окна нет, применяется nextNonOverlappingStart, который сдвигает начало за пересекающиеся интервалы.

Коэффициенты оценки не следует представлять как универсальную оптимизационную модель. Это детерминированная эвристика Luma: при одинаковых входных данных она выбирает одинаковый вариант, отдавая дополнительный вес более раннему времени для важных и сложных дел. Требование «по возможности раньше» тем самым преобразовано в предпочтение, а не в жёсткий запрет позднего размещения. Жёстким ограничением основного пути остаётся вместимость в найденный свободный промежуток; окончательное поведение резервной ветви рассматривается отдельно как ограничение.

Для сложной задачи SubtaskAutoScheduler сначала вычисляет срочность каждой подзадачи из важности и сложности, задаёт ориентир внутри общего периода и последовательно подбирает свободные интервалы. После добавления очередного плана этот интервал становится занятым для следующих подзадач. Родительская задача сохраняется с границами от начала первой до конца последней подзадачи, а каждая подзадача получает parentTaskId. До подтверждения формы черновики подзадач существуют только в памяти интерфейса; после сохранения это отдельные сущности Core Data со структурной связью с родителем.

Построение свободных промежутков показано в листинге 3.2:

swift for interval in busy { if interval.start > cursor { gaps.append(ScheduleInterval(start: cursor, end: interval.start)) } cursor = max(cursor, interval.end) } if cursor < segment.end { gaps.append(ScheduleInterval(start: cursor, end: segment.end)) }

Листинг 3.2 — Выделение свободных промежутков между занятыми интервалами

Этот фрагмент показывает базовый механизм исключения пересечений в основном пути: курсор последовательно перемещается к концу занятого интервала, а кандидаты создаются только в образовавшихся разрывах. Полный алгоритм оценки в основной текст не включается, поскольку его коэффициенты являются авторской эвристикой и лучше воспринимаются в виде краткого пояснения, а не длинного листинга.

После создания обычной задачи AddTaskCoordinator проверяет результат через lateBoundary. Если начало находится до 23:00, а окончание — после этой границы, появляется диалог с вариантами «Оставить как есть» и «Разделить». До выбора положительного действия разделение не выполняется. При согласии первая запись сокращается до 23:00, а остаток создаётся на следующий день с 08:00 и названием с пометкой «продолжение». При отказе сохраняется исходный вариант.

FR-05 подтверждено частично: формы и три вида элементов существуют, но для события короче 15 минут увеличивается расчётная estimatedDuration, тогда как введённая дата окончания не исправляется и запись не отклоняется. FR-06 и FR-07 также подтверждены частично. Основной путь не создаёт пересечений, однако резервный nextNonOverlappingStart не ограничивает результат концом дня и дедлайном; для сложной задачи резервное размещение имеет ту же границу. Сохранение родителя и подзадач не оформлено как единая транзакция.

FR-08 подтверждено частично. Согласие действительно запрашивается, но продолжение связано с первой частью только названием: parentTaskId при его создании равен nil. Сначала обновляется исходная запись, затем отдельной операцией создаётся продолжение; при ошибке второй операции возможен частичный результат. Нужны граничные тесты для длительности 15 минут, ровно 23:00, пустого периода, отсутствия свободного окна, дедлайна и ошибки сохранения. Исходники таких сценарных тестов не обнаружены, а существующий unit-test bundle не запускался.

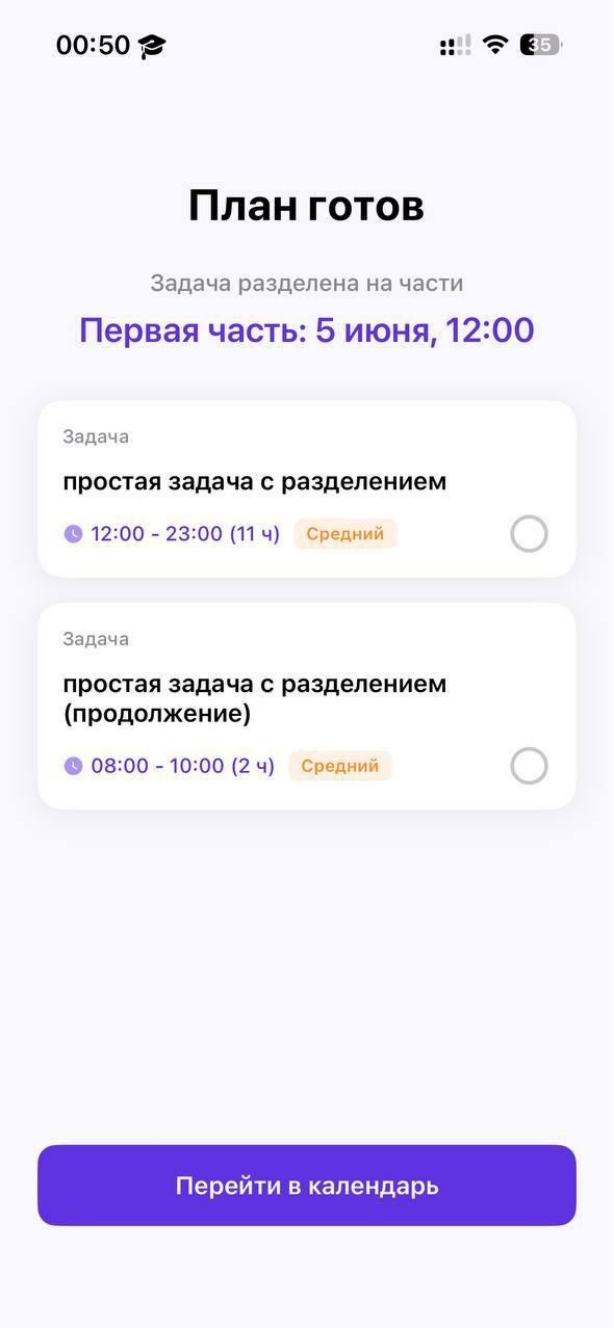


Рисунок 3.2 - Результат разделения задачи при выходе за границу дня

3.4 Уведомления и дневник

FR-09 требует включать уведомления из профиля, запрашивать системное разрешение, сообщать об отказе, создавать напоминания о задачах за 30 и 5 минут, а для сложной задачи — по каждой подзадаче. Для дневника сна заданы моменты 08:00 и 12:00, для дневника эмоций — 18:00 и 20:00; после записи оставшиеся напоминания дня должны отменяться. Доставленные уведомления выводятся отдельным списком в обратной хронологии. FR-10 определяет два вида дневниковых записей и одну выбираемую за операцию эмоцию из перечня: радость, грусть, злость, тревога, покой и любовь. Для эмоции

указываются интенсивность 1–10 и необязательная причина; для сна — время засыпания и пробуждения с автоматическим расчётом длительности.

ProfileViewModel передаёт изменение переключателя в LocalNotificationService. При первом включении сервис запрашивает у UNUserNotificationCenter разрешения alert, sound и badge. Если система возвращает отказ, переключатель сбрасывается, а модель представления выдаёт сообщение о необходимости изменить настройки iOS. При выключении очищаются все ожидающие и все уже доставленные уведомления приложения. Аналогичная полная отмена выполняется при выходе из учётной записи.

Для простой задачи создаются идентификаторы напоминаний на 30 и 5 минут до начала. Сложная задача предварительно раскрывается до подзадач. Если тридцатиминутный момент уже прошёл, такое уведомление не планируется. Если до старта осталось меньше пяти минут, код создаёт второе уведомление примерно через три секунды. Это осознанная резервная ветвь текущей реализации, но она не равна буквальному требованию «за 5 минут» и потому должна быть явно отражена в проверке.

Идентификатор напоминания включает UUID задачи и контрольный интервал, поэтому повторное планирование одного и того же напоминания предварительно удаляет запрос с тем же идентификатором. Это снижает вероятность дубликатов в обычном сценарии создания. Однако изменение времени задачи требует повторного вызова планирования, а удаление — вызова отмены. Само наличие стабильного идентификатора не связывает эти операции автоматически, что и объясняет отмеченное ниже расхождение между доступным методом отмены и фактическим календарным потоком.

Дневниковое расписание строится на семь ближайших дней. Перед добавлением уведомлений сервис проверяет наличие записи соответствующего типа. После успешного сохранения DiaryViewModel вызывает отмену только оставшихся напоминаний эмоций либо сна на текущий день. Экран истории получает доставленные уведомления от системного центра и сортирует их по убыванию даты. В DEBUG-сборке пустой список может быть заменён демонстрационными элементами, поэтому такой экран нельзя считать доказательством фактической доставки без отдельного прогона.

Эмоциональный дневник предоставляет одиночный выбор: EmotionGridView хранит один selectedIndex, а обработчик сохранения формирует один элемент ввода. Перечень в доменной модели точно совпадает с техническим заданием:

swift enum EmotionKind: String, CaseIterable { case joy = "Радость" case sadness = "Грусть" case anger = "Злость" case anxiety = "Тревога" case calm = "Покой" case love = "Любовь" }

Листинг 3.3 — Перечень эмоций, используемый дневником и анализом

Проверка модели показала, что за одно сохранение пользователь выбирает одну эмоцию из шести: радость, грусть, злость, тревогу, покой или любовь. Интенсивность при сохранении ограничивается диапазоном 1–10, причина может отсутствовать. DiaryRepository связывает запись с пользователем и календарным днём. Для сна время относится к выбранному дню, а пробуждение переносится на следующие сутки, если оно не позже времени засыпания; после этого длительность вычисляется в минутах.

FR-10 подтверждено кодом: типы записей, точный перечень, одиночный выбор, интенсивность, необязательная причина, переход сна через полночь и отмена дневниковых напоминаний прослеживаются по вызовам. FR-09 подтверждено частично. Планирование, настройки, история и общая отмена реализованы, но системная доставка и отказ разрешения требуют проверки на устройстве. При редактировании или удалении задачи в CalendarViewModel не найден явный вызов отмены либо перепланирования её старых напоминаний, хотя метод cancelTaskReminders существует. Дополнительной проверки требуют уведомление через три секунды, сортировка реальной истории и отсутствие демонстрационных записей в пользовательской сборке.

3.5 Фокус-сессия и Live Activity

FR-11 предусматривает выбор незавершённой задачи или подзадачи текущего дня, определение продолжительности по плановой оценке, выбор локального плейлиста либо режима без музыки, секундный обратный отсчёт, остановку воспроизведения, сохранение фактической длительности и завершение задачи. На поддерживаемом устройстве Live Activity должна показывать тип и название задачи и оставшееся время на экране блокировки и в Dynamic Island, позволять завершить задачу и закрываться после окончания сеанса. NFR-08 дополнительно требует освобождать таймер, музыку и активность.

FocusViewModel получает элементы текущего дня через тот же TaskService, что и календарь. Сложные задачи раскрываются до незавершённых подзадач. Для выбранного объекта создаётся FocusTaskItem, а продолжительность определяется по estimatedDuration; если она отсутствует, используется разница между концом и началом, с нижней границей одна минута. Локальные плейлисты перечислены в FocusMusicLibrary, воспроизводятся FocusMusicPlayer, а режим без музыки не вызывает проигрыватель.

После старта FocusViewController сохраняет время начала и окончания, запускает Timer с периодом одна секунда и просит FocusLiveActivityManager создать системную активность. Если ActivityKit недоступен или запрос завершился ошибкой, обычный таймер продолжает работу. При завершении контроллер инвалидирует таймер, скрывает карточку отсчёта, завершает Live Activity, останавливает музыку, сохраняет фактический интервал фокус-сессии и отмечает локальную задачу выполненной.

Сохранённая фокус-сессия отделена от плановой длительности задачи. В запись попадают фактические моменты начала и окончания, а репозиторий достижений использует их разницу при пересчёте накопленного времени. Поэтому досрочное завершение не приписывает пользователю всю запланированную продолжительность. Одновременно задача завершается только один раз: при возврате из Live Activity экран получает уведомление и завершает локальное состояние сеанса без повторного изменения задачи.

Расширение Live Activity реализовано на SwiftUI. На экране блокировки оно показывает название и тип задачи, таймер и действие завершения; для Dynamic Island определены развёрнутое, компактное и минимальное состояния. Действие формирует внутреннюю ссылку с идентификатором задачи. AppCoordinator проверяет активную сессию, завершает задачу через TaskService, закрывает активность и сообщает экрану фокуса, чтобы тот не повторял операцию завершения.

На составном рисунке следует показать совпадение названия, типа и оставшегося времени между приложением и системным интерфейсом, а также доступность действия «Завершить задачу».

FR-11 подтверждено частично. Таймер, музыка, запись сессии, код расширения и обработка внутренней ссылки существуют, а приложение и расширение успешно скомпилированы вместе. Однако TaskService.fetchTasks объединяет внутренние задачи с событиями системного календаря, и FocusViewModel не исключает источник .calendar. Такое событие можно выбрать для фокуса, но setCompleted ищет локальную сущность и не может изменить EventKit. Следовательно, системные события не должны описываться как полноценно завершаемые объекты; их необходимо исключить из списка или явно сделать действие завершения недоступным.

Фактическое отображение на экране блокировки, варианты Dynamic Island, переход по ссылке из заблокированного состояния и поведение при сворачивании требуют испытания на поддерживаемом устройстве. Обычный путь завершения останавливает ресурсы, но уничтожение экрана само по себе не вызывает явного завершения Live Activity. Поэтому NFR-08 подтверждено только частично, а отсутствие утечек и корректность принудительного прерывания должны проверяться инструментально.

3.6 Анализ дня, статистика и достижения

FR-12 требует анализировать задачи сегодня и завтра, сон, эмоциональные записи, просрочки и занятое системными событиями время. Результат включает ресурс, нагрузку, баланс, уровень нагрузки, уверенность и не более четырёх рекомендаций. При недостатке данных анализ должен называться предварительным. FR-13 задаёт недельные показатели и

подробную статистику эмоций, сна и задач за соответствующие периоды. FR-14 описывает каталог достижений, состояния locked, unlocked и opened, обновление прогресса по задачам, дневнику и фокус-сессиям, начисление 50 единиц опыта и новый уровень каждые 250 единиц.

AnalysisService собирает пользователя, записи дневника, задачи текущего и следующего дня и передаёт их в UserAnalysisEngine. Сложная задача раскрывается до подзадач. Внешние календарные события участвуют в оценке занятого времени как фиксированные интервалы, но не входят в гибкое усилие пользователя. Отдельно вычисляются текущий и семидневный сон, эмоциональное состояние, оставшиеся и завершённые элементы, просрочки, доступное время и нагрузка следующего дня.

Текущая версия алгоритма не использует прежнее базовое значение 86. Ресурс строится из доступных компонентов: оценки сна с весом 0,65 и эмоционального ресурса с весом 0,35; если обе группы данных отсутствуют, используется нейтральное значение 50. Нагрузка дня рассчитывается из взвешенной длительности гибких задач, уже затраченного усилия, просрочек и фрагментации списка с учётом времени, занятого внешними событиями. Баланс получается из разности ресурса и нагрузки. Значения ограничиваются шкалой 0–100. Уверенность зависит от наличия даты рождения, записи сна, нескольких записей сна за неделю и эмоциональной записи. При низкой уверенности итог помечается как предварительный.

Эмоциональная часть использует тот же EmotionKind, что и дневник. Интенсивность умножается на направление эмоции: радость, покой и любовь поддерживают ресурс, а грусть, злость и тревога увеличивают напряжение. Конкретные веса и пороги являются авторской эвристикой приложения, а не нормативными медицинскими показателями. Модуль формирует пользовательскую подсказку для самоорганизации и не выполняет диагностику состояния здоровья.

Обработку нехватки данных показывает результирующая ветвь анализа:

swift let readinessScore = confidenceLevel == .low ? resourceScore : balanceScore

if confidenceLevel == .low { return "Предварительный анализ: данных пока недостаточно. \(sleepText), задач — \(workText)." }

Листинг 3.4 — Выбор итоговой оценки и маркировка предварительного анализа

Экран общей статистики получает из AnalysisService.weeklyStats количество выполненных и общее количество задач, качество сна, эффективность и эмоциональный показатель за семь дней. Эффективность учитывает длительность, сложность и важность завершённых элементов. Подробные представления показывают 30-дневное распределение эмоций и задач, сведения о сне и состав завершённых и незавершённых задач выбранного

дня. После уведомлений об изменении задач или дневника соответствующие модели представления перезагружают данные.

Для недельных показателей используются сохранённые записи в интервале от начала дня шесть суток назад до начала следующих суток. Качество сна сравнивает длительность с возрастным рекомендуемым диапазоном; эффективность является долей взвешенного объёма завершённых задач; эмоциональный показатель переводит средний баланс из диапазона от −1 до 1 в диапазон от 0 до 1. Эти формулы воспроизводимы по коду, однако их численные результаты должны проверяться заранее подготовленными наборами данных, особенно на границах недельного окна.

AchievementRepository при первом обращении создаёт каталог достижений, затем пересчитывает метрики по завершённым задачам, уникальным дням дневника, продолжительности сна, интенсивным эмоциональным дням и накопленному времени фокуса. Выполненное условие переводит достижение из заблокированного в разблокированное состояние; пользовательское открытие устанавливает opened. AchievementLevelProgress считает только открытые достижения: каждое даёт 50 единиц опыта, каждые 250 единиц повышают уровень, а звание выбирается из заданного ряда. Уровень, звание и доля следующего уровня отображаются в достижениях и профиле.

FR-12 подтверждено кодом. Для UserAnalysisEngine существуют содержательные модульные тесты, охватывающие отсутствие данных, выполнение задач, смешанные эмоции, перегруз, сон, возрастные диапазоны и календарные события. При этом наличие тестовых методов не следует путать с их выполнением в рамках данной проверки: unit-test bundle успешно скомпилирован, но набор не запускался из-за отсутствия Test action в shared-схеме.

FR-13 и FR-14 подтверждены частично. Формулы, экраны и переходы состояний найдены, но недельные и 30-дневные границы, смена часового пояса, повторный пересчёт, границы уровня и численная сверка агрегатов не покрыты подтверждённым прогоном. Для достижений важно различать «разблокировано» и «открыто»: опыт начисляется после открытия, а не сразу при выполнении условия. Это поведение соответствует реализации и должно проверяться отдельными сценариями.

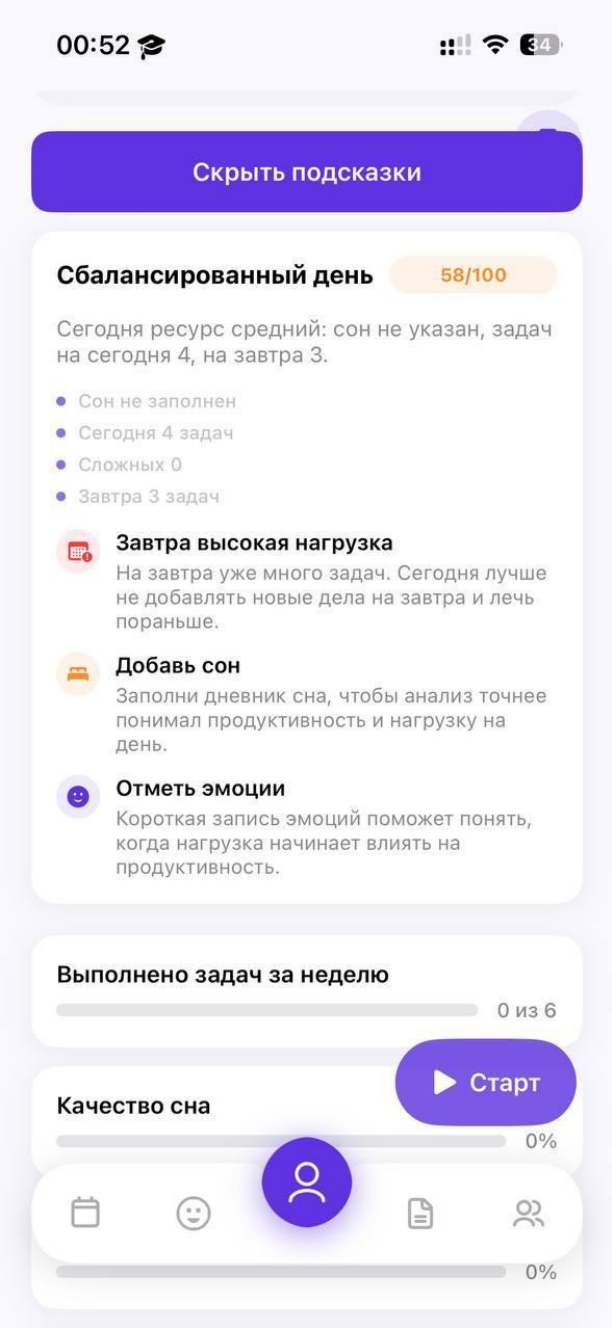


Рисунок 3.3 - Анализ дня и рекомендации

3.7 Трассировка требований и результаты проверки

NFR-10 требует модульной проверяемости логики и автоматизации основных сценариев, а остальные FR и NFR задают конкретные функциональные, аппаратные и количественные критерии. Для итоговой трассировки необходимо различать наличие реализации, успешную компиляцию и фактическое выполнение испытания.

Проверка выполнена по текущему рабочему дереву в трёх формах. Во-первых, проведена статическая инспекция связей между моделями представления, сервисами, репозиториями, доменными алгоритмами и системными API. Во-вторых, выполнена сборка: приложение Luma и расширение Live Activity собраны успешно при отключённой

подписи и командном override ограничения пользовательских скриптов; отдельно успешно скомпилирован unit-test bundle. В-третьих, была предпринята попытка штатного запуска тестов, но shared-схема не настроена для Test action. Поэтому результата выполненного набора тестов нет. Такой исход не означает падение тестов: тестовые методы не запускались.

Проверяемым объектом было текущее состояние рабочего дерева, включая незакоммиченные изменения. Сборочные продукты создавались вне проекта, а исходный код и настройки схемы во время проверки не изменялись. Успешная компиляция подтверждает согласованность типов, подключение расширения и возможность собрать тестовый target в использованной конфигурации. Она не подтверждает корректность алгоритмических результатов, системных разрешений, уведомлений, фонового выполнения и поведения на конкретных устройствах; для этих выводов нужны отдельные протоколы с версией ОС, моделью устройства, входными данными и ожидаемым результатом.

Статусы в таблице 3.1 имеют ограниченный смысл. «Подтверждено кодом» означает, что основная требуемая ветвь явно прослеживается в реализации; это не равнозначно приёмке и не доказывает системное поведение на устройстве. «Частично» означает, что механизм существует, но одна или несколько ветвей расходятся с ТЗ либо требуют сценарного, аппаратного или инструментального испытания. «Требует runtime-испытания» используется, когда статического свидетельства недостаточно для основного вывода.

Таблица 3.1 - Компактная матрица трассировки требований к реализации и проверке

ID	Краткий предмет	Статус	Ключевое свидетельство и граница
FR-01	Регистрация, вход, сессия	Подтверждено кодом	Валидаторы, AuthService, SessionManager; сценарный прогон отсутствует.
FR-02	Профиль, Face ID, аватар, выход	Частично	ProfileViewModel, Face ID, AvatarStore; редактирование учётных данных не реализовано.
FR-03	Календарь, фильтры, прогресс, операции	Подтверждено кодом	CalendarViewModel и TaskService; операции только с собственными задачами.
FR-04	События календаря iOS	Частично	EventKit-сервис чтения; разрешения и обновление требуют runtime-проверки.
FR-05	Событие, простая и сложная задача	Частично	Формы и валидаторы; фактический конец короткого события не исправляется.
FR-06	Планирование обычной задачи	Частично	TaskAutoScheduler; резервный путь не ограничен 23:00 и дедлайном.
FR-07	Планирование подзадач	Частично	SubtaskAutoScheduler; атомарность общего сохранения не подтверждена.

ID	Краткий предмет	Статус	Ключевое свидетельство и граница
FR-08	Согласие и разделение после 23:00	Частично	Диалог и splitTaskAtLateBoundary; связь по названию, сохранение нетранзакционно.
FR-09	Уведомления и история	Частично	LocalNotificationService; перепланирование после изменений не подтверждено.
FR-10	Дневник эмоций и сна	Подтверждено кодом	Одна из шести эмоций, шкала 1–10, сон через полночь.
FR-11	Фокус и Live Activity	Частично	Таймер и Live Activity; внешнее событие ошибочно доступно для выбора.
FR-12	Анализ дня и рекомендации	Подтверждено кодом	AnalysisService и UserAnalysisEngine; тестовые методы не запускались.
FR-13	Недельная и подробная статистика	Частично	weeklyStats и экраны; агрегаты и даты не проверены прогоном.
FR-14	Достижения, опыт и уровни	Частично	AchievementRepository; переходы и границы уровней не проверены прогоном.
NFR-01	Совместимость и деградация возможностей	Требует runtime-испытания	Проверки доступности есть; матрица устройств и версий ОС не пройдена.
NFR-02	Интерфейс и доступность	Частично	Единые компоненты; полный аудит доступности не выполнен.
NFR-03	Отзывчивость и фоновые операции	Частично	Таймер и обработка аватара; количественные замеры отсутствуют.
NFR-04	Надёжность хранения	Частично	Миграция и rollback есть; ошибки сохранения частично подавляются.
NFR-05	Целостность и принадлежность данных	Частично	Связи owner/parentTask есть; многопользовательские и DST-тесты отсутствуют.
NFR-06	Пароль, Keychain, биометрия, выход	Частично	Хеш, соль, Keychain, биометрия; отрицательные ветви не испытаны.
NFR-07	Разрешения и конфиденциальность	Частично	Разрешения запрашиваются по функции; динамическая проверка не выполнена.
NFR-08	Автономность и ресурсы	Частично	Локальные данные и музыка; ресурсные прогоны отсутствуют.
NFR-09	Архитектура и сопровождаемость	Подтверждено кодом	MVVM-C, сервисы, репозитории и протоколы; часть служб - singleton.
NFR-10	Автоматизированные тесты	Частично	Test bundle компилируется; Test action не настроен.

По результатам статической инспекции из четырнадцати функциональных групп четыре подтверждены кодом, а десять подтверждены частично. Из десяти нефункциональных групп одна подтверждена кодом, восемь подтверждены частично, одна требует runtime-испытания. Эти числа характеризуют найденные свидетельства, а не процент готовности и не результат приёмки. Нулевого программного покрытия не обнаружено: для каждой укрупнённой группы существует хотя бы часть механизма, но у составных требований остаются неподтверждённые либо расходящиеся ветви.

Основные экраны готового приложения реализованы, однако остаются ограничения в граничных сценариях и в полноте проверки. Для планирования это резервное размещение вне 23:00 или дедлайна и нетранзакционное разделение; для уведомлений — старые запросы после изменения или удаления задачи; для фокуса — доступность внешнего события, которое нельзя завершить; для хранения — подавление части ошибок; для качественных требований — отсутствие измерений и аппаратной матрицы. Исправление этих пунктов и настройка Test action необходимы для последующего вывода о полном соответствии, но не входят в формулировку уже полученного результата: разработано готовое приложение Luma, приложение и расширение Live Activity успешно компилируются, unit-test bundle компилируется, а выполнение тестов в проведённой проверке не состоялось.

3.8 Пользовательское исследование восприятия интерфейса

После завершения разработки Luma проведено пользовательское исследование восприятия интерфейса. Исследование не являлось функциональным испытанием или приёмкой приложения: оно позволяло оценить субъективное впечатление респондентов от понятности, читаемости, навигации и визуального оформления, но не подтверждало корректность реализации отдельных требований технического задания.

Сбор ответов выполнялся с помощью Google Forms. В исследовании приняли участие 14 респондентов. Участники оценивали готовое приложение Luma. Период проведения, единый набор пользовательских действий, продолжительность знакомства и состав устройств отдельно не фиксировались.

Основная часть анкеты включала утверждения, оцениваемые по пятибалльной шкале: 1 соответствовала минимальной, а 5 — максимальной выраженности оцениваемого признака. Для каждого критерия рассчитаны средняя оценка, число ответов 4–5 и распределение ответов по шкале. Отдельно респонденты оценивали интерфейс по десятибалльной шкале и отвечали на вопрос о готовности продолжить использование приложения. Сводные результаты представлены в таблице 3.2.

Таблица 3.2 - Сводные результаты пользовательского исследования

Критерий	**Средняя оценка**	**Оценки 4–5**	**Распределение ответов 1 / 2 / 3 / 4 / 5**
Понимание назначения приложения	4,29	12 из 14	0 / 1 / 1 / 5 / 7
Понятность структуры главного экрана	4,14	11 из 14	0 / 0 / 3 / 6 / 5
Поиск основных разделов	4,14	11 из 14	0 / 1 / 2 / 5 / 6
Логичность навигации	3,79	9 из 14	0 / 2 / 3 / 5 / 4
Удобство чтения текста	3,86	9 из 14	0 / 1 / 4 / 5 / 4
Визуальная привлекательность дизайна	4,36	11 из 14	0 / 0 / 3 / 3 / 8

Критерий	Средняя оценка	Оценки 4–5	Распределение ответов 1 / 2 / 3 / 4 / 5
Современность дизайна	4,14	10 из 14	0 / 0 / 4 / 4 / 6
Подбор цветов	4,14	10 из 14	0 / 2 / 2 / 2 / 8
Аккуратность отступов, карточек, кнопок и компоновки	4,07	10 из 14	0 / 2 / 2 / 3 / 7
Понятность главных действий на экране	4,14	12 из 14	0 / 1 / 1 / 7 / 5
Удобство форм, календаря, выбора дат и задач	3,86	8 из 14	0 / 1 / 5 / 3 / 5

Наиболее высокую среднюю оценку получила визуальная привлекательность дизайна — 4,36 из 5. Понимание назначения приложения оценено в среднем на 4,29; 12 из 14 участников выбрали значения 4 или 5. Эти ответы показывают, что назначение Luma и общий визуальный образ в основном воспринимались положительно. Распределение оценок понимания назначения целесообразно сохранить в основном тексте как рисунок 3.4.

Более сдержанные оценки получены для логичности навигации — 3,79, удобства чтения текста — 3,86, а также удобства форм, календаря и выбора дат и задач — 3,86. При этом для последнего критерия оценки 4–5 дали 8 из 14 участников. Такое распределение не свидетельствует о непригодности интерфейса, но указывает на направления следующей итерации: упрощение формы создания задачи, повышение заметности выбора даты, уточнение подписей элементов навигации и проверку читаемости текста. Для иллюстрации этой зоны улучшения в основном тексте следует оставить рисунок 3.5.

Средняя общая оценка интерфейса составила 8,21 из 10, медианная — 8,5. На вопрос о готовности продолжить использование 13 из 14 респондентов ответили «Да» или «Скорее да». Эти показатели дополняют оценки отдельных критериев и характеризуют общее впечатление участников, однако не позволяют судить о фактической частоте дальнейшего использования. В основном тексте достаточно привести распределение общей оценки интерфейса на рисунке 3.6.

В основном тексте приведены три диаграммы, отражающие понимание назначения приложения, удобство форм и календаря и общую оценку интерфейса. Первичная выгрузка Google Forms должна храниться вместе с материалами исследования для последующей проверки расчётов.

При интерпретации результатов необходимо учитывать ограничения исследования. Выборка включала 14 человек, поэтому полученные оценки описывают мнение только данной группы и не могут автоматически распространяться на всю целевую аудиторию. Способ набора участников и демографические характеристики не фиксировались, что ограничивает переносимость выводов. Не задокументированы единый набор пользовательских действий, продолжительность знакомства и состав устройств, поэтому

сравнимость условий участия пока не подтверждена. Кроме того, до сверки с первичной выгрузкой Google Forms численные значения рассматриваются как авторская сводка. Исследование выявляет направления улучшения интерфейса, но не заменяет сценарные, аппаратные, функциональные и нефункциональные испытания Luma.

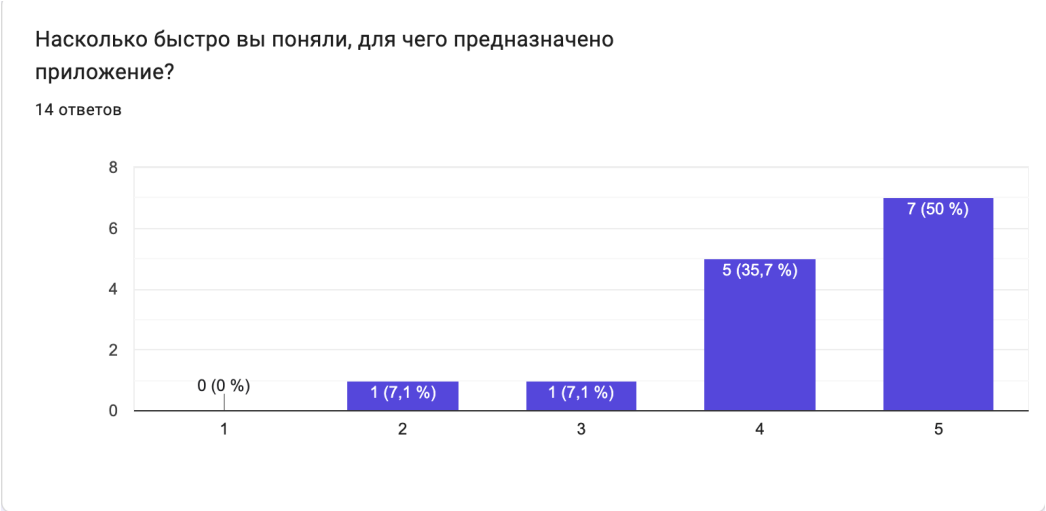


Рисунок 3.4 - Оценка понимания назначения приложения

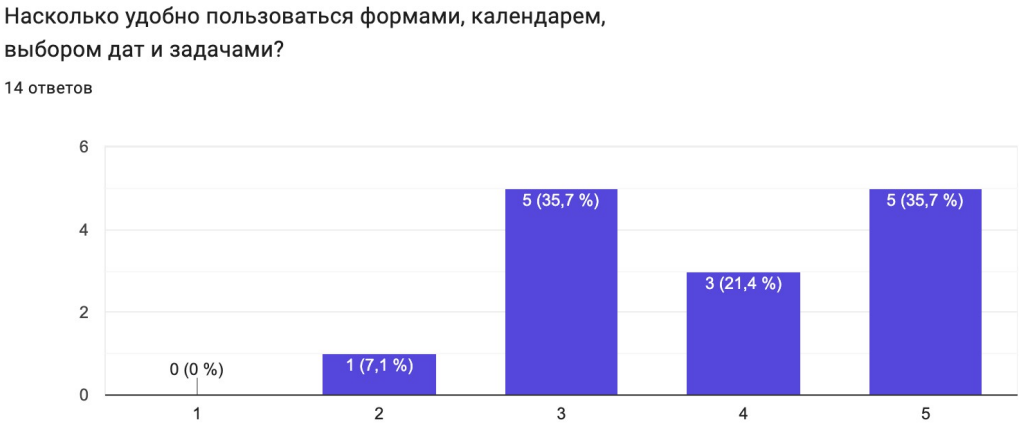


Рисунок 3.5 - Оценка удобства форм, календаря, выбора дат и задач

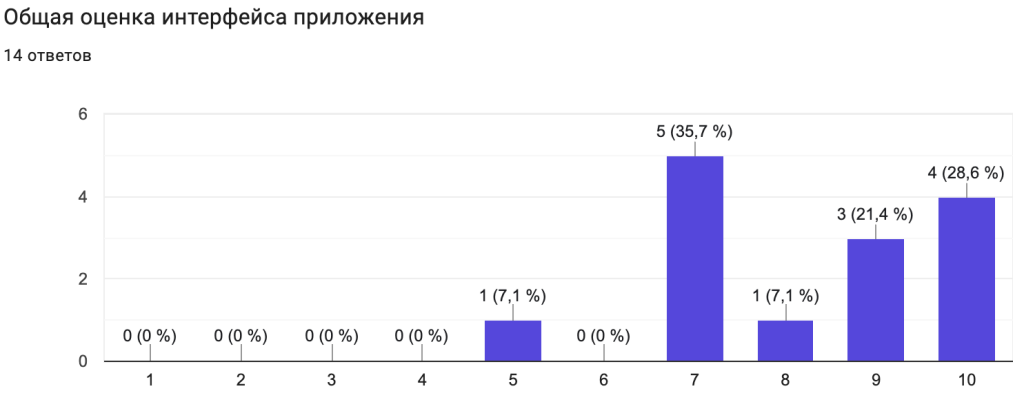


Рисунок 3.6 - Общая оценка интерфейса приложения

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы спроектировано и реализовано готовое iOS-приложение Luma для адаптивного планирования задач, ведения дневника сна и эмоций, проведения фокус-сессий и анализа состояния пользователя. Итоги работы соотносятся с семью задачами, сформулированными во введении.

Для решения первой задачи проанализированы предметная область, предоставленные заказчиком требования и существующие приложения-аналоги. Сравнение показало, что рассмотренные решения поддерживают отдельные функции планирования, самонаблюдения и концентрации, тогда как техническое задание Luma объединяет эти функции в одном приложении.

В ходе решения второй задачи требования формализованы в группы FR-01–FR-14 и NFR-01–NFR-10. Для них определены проверяемые формулировки и способы подтверждения, что позволило связать техническое задание с проектными решениями, программными модулями и необходимыми испытаниями.

По третьей задаче разработаны модели вариантов использования и предметной области. Они зафиксировали границы Luma, цели пользователя, участие системных служб iOS и связи между пользователем, задачами, подзадачами, дневниковыми записями, фокус-сессиями, анализом и достижениями.

Для решения четвёртой задачи обоснован выбор iOS, Swift, UIKit, SwiftUI, Core Data и системных фреймворков. Архитектура MVVM-C разделяет представление, бизнес-логику, навигацию, доступ к данным и интеграции с платформой, создавая основу для сопровождения и дальнейшего тестирования.

Пятая задача включала проектирование пользовательского интерфейса и структуры хранения. Подготовлены экраны основных сценариев, навигация между функциональными разделами и локальная модель данных. Пароль пользователя не хранится в Core Data в открытом виде: его хеш с индивидуальной солью сохраняется в Keychain.

В соответствии с шестой задачей реализованы регистрация и вход, профиль, календарь, создание и планирование задач и подзадач, локальные уведомления, дневник сна и эмоций, фокус-сессия с Live Activity, анализ дня, статистика и система достижений. Статическая инспекция показала наличие программных модулей для всех укрупнённых групп функциональных требований, но также выявила ветви, нуждающиеся в доработке или динамической проверке, в том числе граничные случаи планирования, перепланирование уведомлений, отдельные сценарии Live Activity и обработку ошибок хранения.

По седьмой задаче выполнены предварительная трассировка требований, инспекция исходного кода и пользовательское исследование восприятия интерфейса. Сборка приложения и расширения Live Activity успешно завершена. Отдельная сборка unit-test target также завершена успешно, что подтверждает компиляцию test bundle. При этом выполненного прогона тестовых методов нет: Test action основной shared-схемы не настроен. Поэтому наличие тестового кода нельзя представлять как результат успешного автоматизированного тестирования. Пользовательское исследование с участием 14 респондентов дало среднюю общую оценку интерфейса 8,21 из 10 и обозначило направления улучшения навигации, форм, выбора дат и читаемости; эти данные характеризуют субъективное восприятие и подлежат сверке с выгрузкой Google Forms.

Цель работы достигнута в части проектирования и реализации готового приложения и подготовки комплекта связанных с ним проектных материалов. Однако полученная доказательная база не позволяет утверждать полное соответствие Luma техническому заданию. Для такого вывода необходимо настроить и выполнить автоматизированный прогон, провести сценарные проверки функциональных требований, измерить показатели производительности, проверить совместимость и интерфейс на заявленных устройствах, исследовать доступность, автономную работу и отказоустойчивость, а также документировать результаты приёмки заказчиком. Эти проверки и выявленные по опросу улучшения интерфейса определяют направления дальнейшей работы над приложением.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ И ЛИТЕРАТУРЫ

1. Бегин А. Статистика смартфонов в мире (2025) [Электронный ресурс] // Инклиент. – 2024. – URL: https://inclient.ru/smartphone-stats/ (дата обращения: 26.05.2026). – Текст : электронный.

2. Исследование T-Pay: чем в России друг от друга отличаются владельцы iPhone и смартфонов на Android [Электронный ресурс] // Т-Банк. – 2024. – URL: https://www.tbank.ru/about/news/09122024-t-pay-research-how-do-iphone-and-android-smartphone-owners-differ-from-each-other-in-russia/ (дата обращения: 26.05.2026). – Текст : электронный.

3. Карасева Л. Геймификация: как игровой подход помогает в обучении и на работе [Электронный ресурс] // РБК Тренды. – 2021. – Обновлено: 26.11.2024. – URL: https://trends.rbc.ru/trends/education/605c6f2f9a79473a61646994 (дата обращения: 26.05.2026). – Текст : электронный.

4. Getting started [Электронный ресурс] // Apple Developer Documentation. – URL: https://developer.apple.com/design/human-interface-guidelines/getting-started (дата обращения: 26.05.2026). – Текст : электронный.

5. Филимонов А. П. Анализ современных тенденций развития стратегического планирования [Электронный ресурс] // Экономика, предпринимательство и право. – 2024. – URL: https://cyberleninka.ru/article/n/analiz-sovremennyh-tendentsiy-razvitiya-strategicheskogo-planirovaniya (дата обращения: 03.06.2026). – Текст : электронный.

6. Назарова Е. К. Влияние психики на производительность труда [Электронный ресурс] // Universum: психология и образование. – 2024. – № 7(121). – URL: https://cyberleninka.ru/article/n/vliyanie-psihiki-na-proizvoditelnost-truda (дата обращения: 03.06.2026). – Текст : электронный.

7. Новикова Ю. Л., Аулова О. Ю., Быковская Е. С. Физиология сна и его влияние на психоэмоциональное состояние студентов [Электронный ресурс] // Вестник МГПУ. Серия «Естественные науки». – 2023. – № 2(50). – DOI: 10.25688/2076-9091.2023.50.2.04. – URL: https://iest-vestnik.mgpu.ru/wp-content/uploads/sites/5/2023/08/novikova-yu-l-aulova-o-yu-bykovskaya-e-s.pdf (дата обращения: 03.06.2026). – Текст : электронный.

8. Morrissette M. New study shows treating work like a game drives results [Электронный ресурс] // Phys.org. – 2024. – 25 March. – URL: https://phys.org/news/2024-03-game-results.html (дата обращения: 03.06.2026). – Текст : электронный.

9. Aeon B., Faber A., Panaccio A. Does time management work? A meta-analysis [Электронный ресурс] // PLOS ONE. – 2021. – Vol. 16, № 1. – e0245066. – DOI: 10.1371/journal.pone.0245066. – URL:

https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0245066 (дата обращения: 03.06.2026). – Текст : электронный.

10. Takano Y., Iwano S., Aoki S. et al. A systematic review of the effect of sleep interventions on presenteeism [Электронный ресурс] // BioPsychoSocial Medicine. – 2021. – Vol. 15, Article 21. – DOI: 10.1186/s13030-021-00224-z. – URL: https://link.springer.com/article/10.1186/s13030-021-00224-z (дата обращения: 03.06.2026). – Текст : электронный.

11. Weiss H. M., Merlo K. L. Affect, Attention, and Episodic Performance [Электронный ресурс] // Current Directions in Psychological Science. – 2020. – Vol. 29, № 5. – P. 453–459. – DOI: 10.1177/0963721420949496. – URL: https://journals.sagepub.com/doi/10.1177/0963721420949496 (дата обращения: 03.06.2026). – Текст : электронный.

12. Oliveira M., Abelha A., Sousa R., Peixoto H. Gamification in mobile applications: techniques, benefits and challenges [Электронный ресурс] // Procedia Computer Science. – 2024. – Vol. 251. – P. 678–683. – DOI: 10.1016/j.procs.2024.11.168. – URL: https://www.sciencedirect.com/science/article/pii/S1877050924034021 (дата обращения: 03.06.2026). – Текст : электронный.

13. Paschmann J. W., Bruno H. A., van Heerde H. J., Völckner F., Klein K. Driving Mobile App User Engagement Through Gamification [Электронный ресурс] // Journal of Marketing Research. – 2025. – Vol. 62, № 2. – P. 249–273. – DOI: 10.1177/00222437241275927. – URL: https://journals.sagepub.com/doi/10.1177/00222437241275927 (дата обращения: 03.06.2026). – Текст : электронный.

14. Oakley-Girvan I., Yunis R., Longmire M., Ouillon J. S. What Works Best to Engage Participants in Mobile App Interventions and e-Health: A Scoping Review [Электронный ресурс] // Telemedicine and e-Health. – 2022. – DOI: 10.1089/tmj.2021.0176. – URL: https://journals.sagepub.com/doi/10.1089/tmj.2021.0176 (дата обращения: 03.06.2026). – Текст : электронный.

15. Karlsen F. Affordances in digital detox and productivity apps [Электронный ресурс] // AoIR Selected Papers of Internet Research. – 2021. – DOI: 10.5210/spir.v2021i0.11956. – URL: https://spir.aoir.org/ojs/index.php/spir/article/view/11956 (дата обращения: 03.06.2026). – Текст : электронный.

16. Weichbroth P. Usability Testing of Mobile Applications: A Methodological Framework [Электронный ресурс] // Applied Sciences. – 2024. – Vol. 14, № 5. – Article 1792. – DOI: 10.3390/app14051792. – URL: https://www.mdpi.com/2076-3417/14/5/1792 (дата обращения: 03.06.2026). – Текст : электронный.

17. Milne-Ives M., Homer S. R., Andrade J., Meinert E. Potential associations between behavior change techniques and engagement with mobile health apps: a systematic review [Электронный ресурс] // Frontiers in Psychology. – 2023. – Vol. 14. – Article 1227443. – DOI: 10.3389/fpsyg.2023.1227443. – URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC10545861/ (дата обращения: 03.06.2026). – Текст : электронный.

18. Matthews P., Rhodes-Maquaire C. Personalisation and Recommendation for Mental Health Apps: A Scoping Review [Электронный ресурс] // Behaviour & Information Technology. – 2025. – Vol. 44, № 10. – P. 2389–2404. – DOI: 10.1080/0144929X.2024.2356630. – URL: https://www.tandfonline.com/doi/full/10.1080/0144929X.2024.2356630 (дата обращения: 03.06.2026). – Текст : электронный.

19. Apple Inc. Model-View-Controller [Электронный ресурс] // Apple Developer Documentation Archive. – 2018. – URL: https://developer.apple.com/library/archive/documentation/General/Conceptual/DevPedia-CocoaCore/MVC.html (дата обращения: 21.06.2026). – Текст : электронный.

20. Fowler M. Presentation Model [Электронный ресурс] // martinfowler.com. – 2004. – URL: https://martinfowler.com/eaaDev/PresentationModel.html (дата обращения: 21.06.2026). – Текст : электронный.

21. Model-View-ViewModel (MVVM) [Электронный ресурс] // Microsoft Learn. – URL: https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm (дата обращения: 21.06.2026). – Текст : электронный.

22. Khanlou S. Coordinators Redux [Электронный ресурс] // Khanlou. – 2015. – URL: https://khanlou.com/2015/10/coordinators-redux/ (дата обращения: 21.06.2026). – Текст : электронный.

23. Martin R. C. The Clean Architecture [Электронный ресурс] // The Clean Code Blog. – 2012. – URL: https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html (дата обращения: 21.06.2026). – Текст : электронный.

24. Gilbert J., Stoll C. Architecting iOS Apps with VIPER [Электронный ресурс] // objc.io. – 2014. – URL: https://www.objc.io/issues/13-architecture/viper/ (дата обращения: 21.06.2026). – Текст : электронный.

25. TickTick: A To-Do List and Calendar to Keep You Organized [Электронный ресурс] // TickTick. - URL: https://www.ticktick.com/ (дата обращения: 25.09.2026). - Текст : электронный.

26. Structured: Daily Planner [Электронный ресурс] // Structured. - URL: https://structured.app/ (дата обращения: 25.09.2026). - Текст : электронный.

27. How to Create, Edit & Delete Tasks [Электронный ресурс] // Structured Help Center. - URL: https://help.structured.app/en/articles/338050 (дата обращения: 25.09.2026). - Текст : электронный.

28. Simplify Task Management with Todoist [Электронный ресурс] // Todoist. - URL: https://www.todoist.com/task-management (дата обращения: 25.09.2026). - Текст : электронный.

29. The Fabulous: Build Better Habits & Achieve Your Goals [Электронный ресурс] // Fabulous. - URL: https://www.thefabulous.co/ (дата обращения: 25.09.2026). - Текст : электронный.

30. How Does Fabulous Work? [Электронный ресурс] // The Fabulous Help Center. - URL: https://help.thefabulous.co/en/support/solutions/articles/101000427430-how-does-fabulous-work- (дата обращения: 25.09.2026). - Текст : электронный.

ПРИЛОЖЕНИЕ А

Дополнительные экраны приложения

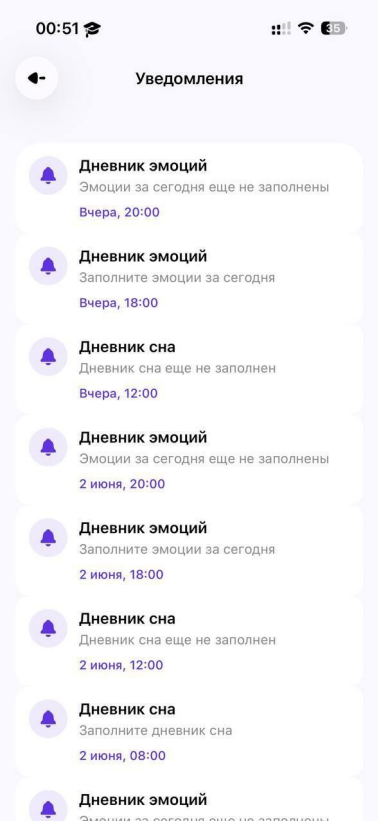


Рисунок А.1 - История уведомлений

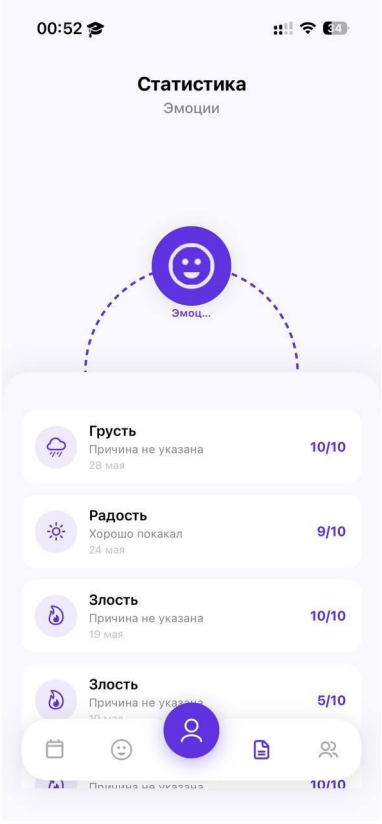


Рисунок А.2 - Статистика эмоционального состояния

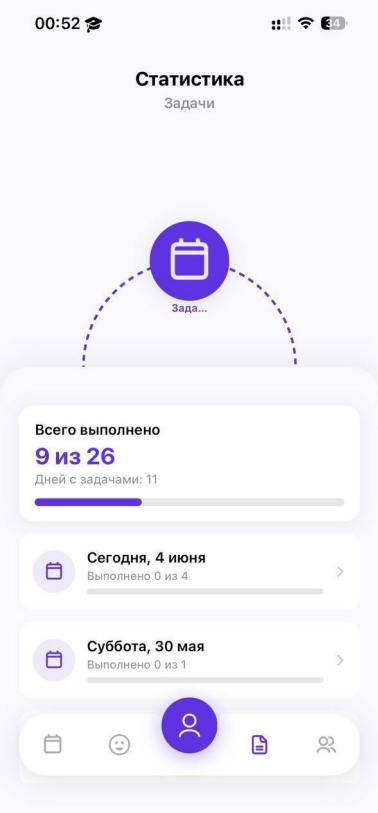


Рисунок А.3 - Статистика выполнения задач



Рисунок А.4 - Статистика сна